

Titel der Arbeit

Optionaler Untertitel der Arbeit

BACHELORARBEIT

zur Erlangung des akademischen Grades

Bachelor of Science

im Rahmen des Studiums

Informatik mit Vertiefung Digital Health

eingereicht von

Sebastian Bolek

Matrikelnummer 12323843

an der Fakultät für Informatik

der Technischen Universität Wien

Betreuung: Associate Prof. Dr.in Renata Georgia Raidou

Mitwirkung: Pretitle Forename Surname, Posttitle

Pretitle Forename Surname, Posttitle

Pretitle Forename Surname, Posttitle

Wien, 1. Jänner 2001

Sebastian Bolek

Renata Georgia Raidou

Title of the Thesis

Optional Subtitle of the Thesis

BACHELOR'S THESIS

submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science

in

Informatics with specialization in Digital Health

by

Sebastian Bolek

Registration Number 12323843

to the Faculty of Informatics

at the TU Wien

Advisor: Associate Prof. Dr.in Renata Georgia Raidou

Assistance: Pretitle Forename Surname, Posttitle

Pretitle Forename Surname, Posttitle

Pretitle Forename Surname, Posttitle

Vienna, January 1, 2001

Sebastian Bolek

Renata Georgia Raidou

Declaration of Authorship

Sebastian Bolek

I hereby declare that I have written this thesis independently, that I have completely specified the utilized sources and resources and that I have definitely marked all parts of the work – including tables, maps and figures – which belong to other works or to the internet, literally or extracted, by referencing the source as borrowed.

I further declare that I have used generative AI tools only as an aid, and that my own intellectual and creative efforts predominate in this work. In the appendix “Overview of Generative AI Tools Used” I have listed all generative AI tools that were used in the creation of this work, and indicated where in the work they were used. If whole passages of text were used without substantial changes, I have indicated the input (prompts) I formulated and the IT application used with its product name and version number/date.

Vienna, January 1, 2001

Sebastian Bolek

Acknowledgements

Enter your text here.

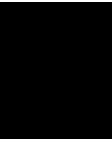
Abstract

Enter your text here.

Contents

Abstract	ix
Contents	xi
1 Introduction	1
2 Related Work	3
3 Methodology	5
3.1 Overview of the approach	5
3.2 Input data	5
3.3 Choosing how to recover thickness	6
3.4 Aligning the mesh and the volume	7
3.5 Measuring leaflet thickness in 3D Slicer	9
3.6 Spreading the measurements over the mesh	10
3.7 Building the solid model	15
3.8 Printing the complete model	15
4 Results	19
5 Conclusion	21
Overview of Generative AI Tools Used	23
List of Figures	25
List of Tables	27
List of Algorithms	29
Bibliography	31

CHAPTER 1



Introduction

CHAPTER 2

Related Work

Methodology

3.1 Overview of the approach

The goal of this work is a 3D-printed prototype of a thickness-accurate 3D model of the valve leaflets that can serve as the basis for an ultrasound phantom. The starting point is two things: a volumetric ultrasound scan of the valve, stored as a VDB volume, and a surface mesh of the leaflets. The problem is that these two do not directly provide what is needed. The scan contains the full valve, but only as a grid of grayscale values with no explicit geometry. The mesh has clean geometry, but it is only a single surface with no thickness, so on its own it cannot be turned into a physical object with a wall.

The core task of the whole pipeline is therefore to bridge these two: to bring the mesh and the volume into a common coordinate system, to recover the leaflet thickness from the volume, to transfer that thickness to the mesh, and to turn the single surface into a closed solid.

I did not carry out all of this in a single program, because no single tool does every part well. Instead the work is split across three: MeVisLab is used to load and prepare the volume, 3D Slicer is used to align the mesh with the volume and to take the thickness measurements, and Blender is used to interpolate the measurements over the mesh and build the solid. Some smaller steps, mainly the interpolation, were done with short Python scripts using the numpy library. The following sections describe each part, and, more importantly, why each choice was made and not one of the other possible ones.

3.2 Input data

The first input is the volumetric scan. It is a 3D grayscale field where bright voxels roughly correspond to tissue and dark voxels to the surrounding space. It contains the leaflets with their real thickness, but this thickness is only present as a bright band in

the image and not as a number or a shape that can be measured automatically without further steps. The scan is also not clean everywhere: the ultrasound probe used to record it left a very noisy region in the centre of the valve, around the opening, which becomes important later.

The second input is the surface mesh of the leaflets. This mesh is a single surface. It does not enclose a volume, and it has no wall thickness at all. In practice there are two separate leaflet meshes, referred to here as [Mesh1] and [Mesh2].

Before deciding how to add thickness, I first had to establish what the surface actually represents, because this changes the whole calculation. A surface mesh of a thin structure can either be a *midsurface*, sitting in the middle between the two walls, or a *boundary surface*, sitting on one side. If it is a midsurface, the real thickness is split evenly on both sides of it. If it is a boundary surface, the whole thickness sits on one side only. Getting this wrong would make every thickness value off by a factor of two.

To check this, the mesh was placed next to the volume and a cross-section through a leaflet was examined. If the mesh were a midsurface, it would run through the middle of the bright leaflet band. In this case the surface sits on one side of the band, which means it is the underside (a boundary surface), not the middle. This has a direct consequence for the later steps: the full measured thickness has to be added on one side, and there is no division by two. I come back to this in Section 3.7.

3.3 Choosing how to recover thickness

Thickness is not stored anywhere in the data. It has to be recovered from the grayscale volume. There were two reasonable ways to do this, and it is worth explaining both, because the reason for choosing the second one is part of the method.

The first idea was to compute the thickness fully automatically with a distance transform. The steps would be: turn the volume into a binary mask, where material is 1 and everything else is 0, and then run a distance transform on that mask. A distance transform gives, for every inside voxel, the distance to the nearest background voxel, in other words the distance to the closest wall [edM]. In the middle of a leaflet, that distance is half the local thickness, so the thickness could in principle be read off as two times the distance value. This was set up in MeVisLab using an `OpenVDBLoad` module to load the volume, a `Threshold` module to make the binary mask (values above 0.2 set to 1, everything else to 0), and a `EuclideanDistanceTransform` module in 3D mode to produce the distance field.

The reason this idea is attractive is that it is automatic and dense: it would give a thickness value everywhere on the leaflet at once, without any manual work. In the end, though, I decided not to use it, for a few reasons that add up.

First, there is a practical problem with reading the result. To get thickness at the mesh, the distance field has to be sampled exactly at the positions of the mesh points. MeVisLab

does not have a simple, ready-made module for “sample this volume at the vertices of that mesh”, and the scripting module has no image input or output that would let this be done directly inside the network. It would mean exporting the distance field and doing the sampling externally in Python, which is possible but adds another step that would itself have to be checked.

Second, the simple “two times the distance” rule only holds if the mesh is a midsurface. As shown in Section 3.2, the mesh here is the underside. For a boundary surface the correct value would be the distance from the underside across to the top wall, which is not what a symmetric distance transform gives without extra work. So the clean version of the method does not actually match this data.

Third, and most importantly, the whole result depends on one global threshold that separates material from background, and the volume is not clean enough for that. The Ultrasound volume has speckled and blurred edges, and this scan has an especially noisy region in the centre of the valve. A single fixed threshold applied to the whole volume would turn that noise into false geometry exactly where the data is worst, and there would be no easy way to notice that it had happened. A dense automatic field of numbers is only useful if it can be trusted, and here it could not be.

Taken together, these points describe a method that would produce a lot of numbers that could not be properly checked. For a bachelor’s thesis, a dense automatic result that cannot be verified is worse than a smaller set of values that can. This is the reason I chose to measure the thickness by hand at a limited number of well-imaged locations instead. Measuring by hand, however, only works if the mesh is correctly placed inside the volume, so the first step before measuring was to align the two. This is described next.

3.4 Aligning the mesh and the volume

The mesh and the volume did not overlap when they were first loaded together in both MeVisLab and 3D Slicer - only Blender could directly open the .usda file which preserved both coordinate systems in the correct orientation. In MeVisLab and 3D Slicer they sat in different coordinate systems. This alignment matters here because, as described in Section 3.5, the thickness is measured along lines perpendicular to the mesh surface. If the mesh is in the wrong place, those perpendicular lines start in the wrong place, and the measurement is meaningless. So the mesh and the volume had to be brought into the same coordinate frame first. Before performing any geometric operations in Blender, all input data was uniformly scaled up by 750%. This initial upscaling was applied to improve numerical precision during computational and transformation steps within Blender’s modeling environment (as further discussed in Section 3.6) and resolved export scaling issues, ensuring the resulting STL file imported at a realistic scale in the 3D-printing slicing software. This uniform scaling is fully tolerable for the valve model, as its final absolute dimensions are ultimately constrained by the physical dimensions

of the ventricular chambers developed by Machoritsch, allowing the valve's scale to be adjusted accordingly prior to final production.

Aligning the two by their shapes directly is difficult, because the leaflet surface is smooth and curved and offers no clear matching landmarks to line up by eye. Instead, I used a reference that all three programs agree on: the bounding box. MeVisLab, 3D Slicer and Blender each draw an axis-aligned bounding box around the volume, and this box is the same in all of them because it is derived from the same volume. Since everything was perfectly aligned in Blender I added planes to the mesh that correspond to the bounding box of the volume, so that the mesh carries the same box as a built-in reference. The resulting mesh is depicted in Figure 3.1. Matching the mesh's bounding planes to the volume's bounding box then defines the alignment completely, because the two boxes have to coincide.

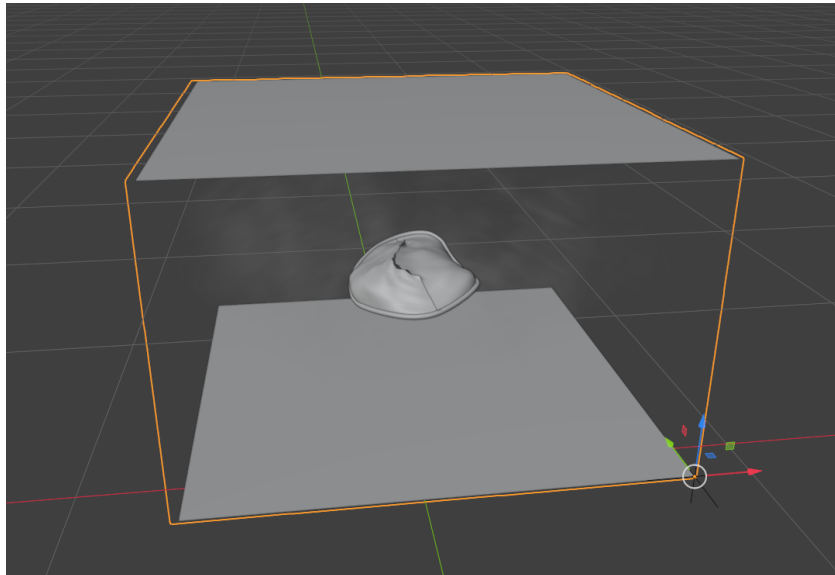


Figure 3.1: Planes added to the mesh in Blender to correspond to the bounding box of the volume.

From this box-to-box correspondence a transformation matrix was obtained, which was applied to the mesh in 3D Slicer. After applying it, the mesh sits inside the volume in the correct position and orientation, so that a leaflet in the mesh lines up with the corresponding bright band in the scan. Using the bounding box as a shared reference is more reliable here than trying to register the curved surfaces against each other, and it keeps the alignment reproducible, because the box is defined by the data and not by a subjective judgement of where the surfaces should meet.

3.5 Measuring leaflet thickness in 3D Slicer

With the mesh aligned, the thickness was measured in 3D Slicer [FBKC⁺12]. The tool used is the line (ruler) markup, which gives a direct distance reading in millimetres between two points. The important part is not the tool but the rule I followed for placing the two points, because a clear rule is what makes the measurements repeatable.

I chose not to judge the leaflet edges purely by eye, and I also chose not to threshold the whole volume, for the reasons given in Section 3.3. Instead, I used a local intensity criterion, applied only at the spot being measured. Tissue was defined as an intensity of 0.2 or higher. Each measurement line was placed perpendicular to the mesh surface at the chosen location. Along that line, the measurement starts where the intensity first rises to 0.2 and ends where it falls back on or below 0.2, and the thickness is the length of that span. This is the same kind of intensity criterion that the automatic approach would have used, but applying it locally, along a single line at a spot I had judged to be clean, avoids the problem of applying it to the whole noisy volume at once. I chose where to place each line; the intensity rule then decided where the tissue began and ended.

Where the measurements were taken was itself a choice driven by the data quality. Thickness is not the same everywhere on a leaflet, so I wanted to sample across the surface, but two regions turned out to be hard to measure. The first is the centre of the valve, around the opening, where the probe left a noisy hole in the scan. In that region it is difficult to see a clean tissue band, so reliable perpendicular measurements are hard to obtain. The second is the thin outer brim of the leaflet, at the edge of the mesh, where measurements were also difficult to take cleanly. In between these two problem regions the data is clearer, so most of the measurements were taken there, where the tissue band is well defined and the intensity rule can be applied with confidence. The resulting locations where measurements were taken are shown in Figure 3.2 although the numbers in the figure do not correspond to the actual indexing of the measurements in Table 3.1. *It should be mentioned that the Aortic valve is shown as well in the figure and it was within the scope of the project to produce a model of the Aortic valve too, but due to the poor quality of the ultrasound data around it there were nearly any areas where measurements could be taken. Therefore, the Aortic valve was not included in the final model.*

For the two difficult regions I made a deliberate simplification rather than forcing measurements the data does not support. For the noisy centre I took all the measurements I could obtain there and reduced them to a single value, the median, and treated the thickness across the whole centre region as constant at that value. I did the same for the outer edge, taking the median of the edge measurements and treating the edge as having one constant thickness. The median was used rather than the mean because it is less affected by the occasional bad reading, which is exactly what one expects in these noisy regions. Assuming a homogeneous thickness in the centre and at the edge is a simplification, and it is stated as such, but it is a reasonable one: the data in those regions is not good enough to justify anything finer, and a single well-chosen value is

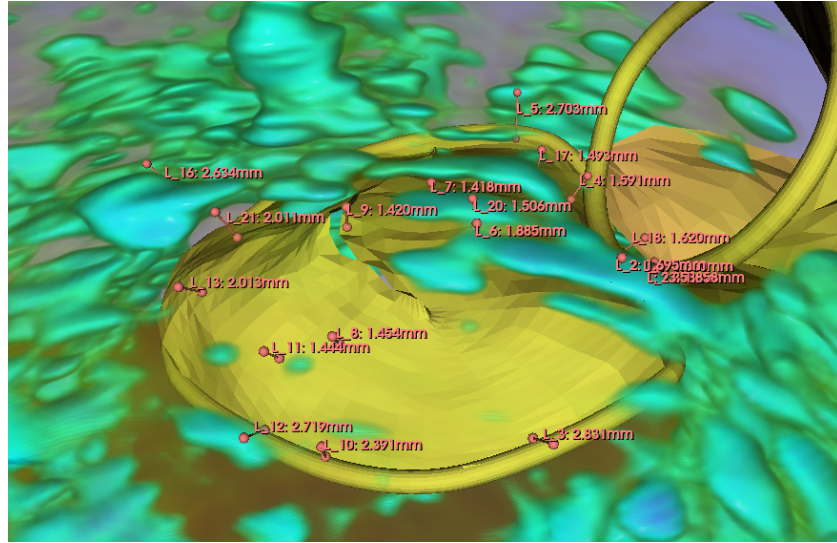


Figure 3.2: Locations of the manual thickness measurements on the leaflets meshes.

more honest than a set of unreliable point measurements.

3.6 Spreading the measurements over the mesh

At this point there is, for each leaflet, a set of thickness values: several point measurements in the well-imaged middle region, plus one constant value for the noisy centre and one constant value for the outer edge. The mesh, however, has several thousand vertices, and the step that builds the solid (Section 3.7) needs a thickness value at every single vertex. So the measured values have to be spread over the whole surface.

The measured points can be recorded as follows in Table 3.1, which lists the coordinates of each measured point and the corresponding thickness value. The table is split into two parts, one for each leaflet mesh, and it includes the sample points, the outer rim points, and the centre points. The outer rim and centre points are treated as having constant thickness values, as described in Section 3.5.

The simplest way to fill the gap would be to give the whole leaflet one average thickness. I rejected this because it would remove the variation between the base and the edge that I went to the trouble of capturing. So some form of interpolation is needed for the region that is well sampled, while the centre and the edge keep their constant median values as described in Section 3.5.

For the interpolation I chose inverse distance weighting (IDW) [She68]. The idea is easy to state: each vertex is given a thickness that is a weighted average of the known values, and values that are closer to the vertex count more than ones further away. The weight of each value falls off with distance, following one over the distance raised to a power. A

Point	Location	Leaflet	Coordinates (X, Y, Z) [mm]	Thickness [mm]
Mesh1				
1	Sample 1	Mesh1	(63.9590, 1.2938, -13.9180)	1.858
2	Sample 2	Mesh1	(64.5597, 0.8593, -11.7677)	1.695
3	Sample 3	Mesh1	(58.5578, -7.5698, -8.4560)	2.831
4	Sample 4	Mesh1	(65.7383, 16.3467, -7.1801)	1.493
5	Sample 5	Mesh1	(64.7657, 14.2849, -9.8768)	1.591
6	Sample 6	Mesh1	(65.3708, 26.0424, -4.8703)	2.703
7	Sample 7	Mesh1	(64.9545, 2.3063, -3.0277)	1.885
8	Sample 8	Mesh1	(66.2014, 7.9130, 0.3569)	1.418
9	Sample 9	Mesh1	(65.9559, 3.1092, 5.0442)	1.420
10	Sample 10	Mesh1	(62.5210, 8.6762, -13.1107)	1.660
11	Outer Rim 1	Mesh1	(60.0724, -5.3534, -13.0215)	2.634
12	Outer Rim 2	Mesh1	(60.8634, 2.2560, -16.6014)	2.634
13	Outer Rim 3	Mesh1	(59.9938, 8.3517, -15.4400)	2.634
14	Outer Rim 4	Mesh1	(60.5665, 15.1949, -13.5914)	2.634
15	Outer Rim 5	Mesh1	(62.8466, 20.5929, -11.7375)	2.634
16	Outer Rim 6	Mesh1	(65.2000, 25.0223, -9.6075)	2.634
17	Outer Rim 7	Mesh1	(65.8712, 27.4709, -6.3174)	2.634
18	Outer Rim 8	Mesh1	(64.6092, 29.0350, 0.8741)	2.634
19	Outer Rim 9	Mesh1	(59.9680, 11.1331, -14.7723)	2.634
20	Center 1	Mesh1	(63.6521, -0.7522, -0.0410)	1.420
21	Center 2	Mesh1	(66.8656, 9.9355, 4.1078)	1.420
22	Center 3	Mesh1	(66.5859, 16.3126, 2.0336)	1.420
23	Center 4	Mesh1	(66.5978, 20.5522, 0.9076)	1.420
Mesh2				
24	Sample 11	Mesh2	(55.8837, -7.7487, 3.9077)	2.391
25	Sample 12	Mesh2	(56.0970, -4.6908, 8.6596)	2.719
26	Sample 13	Mesh2	(59.9952, -3.0386, 6.9769)	1.444
27	Sample 14	Mesh2	(60.6347, -3.7250, 2.8164)	1.454
28	Sample 15	Mesh2	(63.1115, 3.0225, 11.4841)	2.013
29	Sample 16	Mesh2	(65.1467, 5.8269, 8.7319)	2.011
30	Sample 17	Mesh2	(60.3688, 16.5809, 15.3959)	2.634
31	Outer Rim 1	Mesh2	(63.7587, 28.0507, 2.7245)	2.634
32	Outer Rim 2	Mesh2	(63.4924, 25.6784, 7.3205)	2.634
33	Outer Rim 3	Mesh2	(61.9454, 21.5032, 11.7638)	2.634
34	Outer Rim 4	Mesh2	(60.0886, 15.0826, 16.3647)	2.634
35	Outer Rim 5	Mesh2	(58.0204, 8.4046, 18.1553)	2.634
36	Outer Rim 6	Mesh2	(56.3447, 3.7394, 17.3366)	2.634
37	Outer Rim 7	Mesh2	(55.1820, -0.8097, 13.2220)	2.634
38	Outer Rim 8	Mesh2	(55.8221, -6.6502, 5.8992)	2.634
39	Outer Rim 9	Mesh2	(56.0250, -8.6121, 1.7725)	2.634
40	Outer Rim 10	Mesh2	(56.9056, -8.9694, -3.2529)	2.634
41	Center 1	Mesh2	(63.7351, -1.3024, 0.4987)	1.420
42	Center 2	Mesh2	(64.8664, 1.7162, 5.0023)	1.420
43	Center 3	Mesh2	(66.8032, 5.1267, 6.6123)	1.420
44	Center 4	Mesh2	(66.8336, 9.7100, 4.8804)	1.420
45	Center 5	Mesh2	(66.7762, 14.3475, 3.3624)	1.420
46	Center 6	Mesh2	(66.1239, 20.7572, 1.0062)	1.420

Table 3.1: Applied leaflet thickness and 3D spatial coordinates from the Blender model at all sampled locations, outer rim points, and center points for Mesh1 and Mesh2.

vertex sitting right next to a known value gets almost exactly that value, and a vertex halfway between two of them gets roughly their average.

I chose IDW over more advanced interpolation methods such as kriging or radial basis functions on purpose. With only a handful of known values, the more advanced methods bring in extra assumptions and parameters that could not be checked with so little data. IDW, in contrast, makes no assumption about an underlying function, works with irregularly placed values, and is simple enough to explain and defend in full. For the power parameter, a value of two was used. This gives greater weight to nearby measurements while still providing a smooth transition between the measured points. The resulting assignment of thickness values to the mesh is shown in Figure 3.3. The outer brim of the leaflet is shown in red as it has the maximum thickness, while the centre of the leaflet, the thinnest part, is shown here in green which is roughly half of the maximum thickness and therefore not in blue, as blue would map to a thickness of zero. The transition between the two is smooth, as expected from the interpolation.

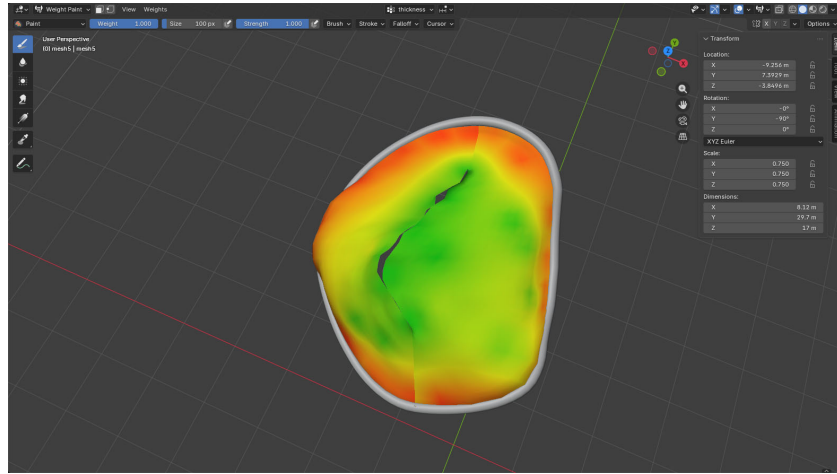


Figure 3.3: Top-down view of the leaflet mesh in Blender, showing the interpolated thickness values as a vertex group. The color gradient represents the thickness, with red indicating thicker regions and blue indicating thinner regions.

The interpolation itself was carried out in Blender using a short Python script, as shown in Listing 3.1 and the values and coordinates as in Table 3.1. NumPy [HMVDW⁺20] is used for the distance and weighted averaging calculations. For each vertex, the script calculates the Euclidean distance to all measured points, determines the corresponding inverse-distance weights, and computes the weighted average of the measured thickness values. For every vertex the script computes the distances to the known values, forms the weights, and takes the weighted average. The result is stored on the mesh as a per-vertex weight, called a vertex group in Blender. Before storing, the values are scaled to the range zero to one by dividing by the largest thickness, because the modifier used in the next step expects weights in that range.


```

1 import bpy
2 import numpy as np
3
4
5 def apply_thickness(obj_name, known_pos, known_val, power=2):
6     obj = bpy.data.objects[obj_name]
7     mesh = obj.data
8
9     all_verts = np.array([v.co for v in mesh.vertices])
10
11     def idw(query, kpos, kval, power):
12         diff = query[:, None, :] - kpos[None, :, :]
13         dist = np.maximum(np.linalg.norm(diff, axis=2), 1e-9)
14
15         w = 1.0 / dist**power
16
17         return (w * kval).sum(1) / w.sum(1)
18
19     thickness_mm = idw(
20         all_verts,
21         np.array(known_pos),
22         np.array(known_val),
23         power
24     )
25
26     max_d = np.array(known_val).max()
27
28     # Remove existing group for re-runs
29     if "thickness" in obj.vertex_groups:
30         obj.vertex_groups.remove(
31             obj.vertex_groups["thickness"]
32         )
33
34     vg = obj.vertex_groups.new(name="thickness")
35
36     for i, d in enumerate(thickness_mm):
37         vg.add(
38             [i],
39             float(d / max_d),
40             'REPLACE'
41         )
42
43     print(
44         f"{obj_name}: "
45         f"{thickness_mm.min():.2f} - "
46         f"{thickness_mm.max():.2f} mm "
47         f"(max={max_d}) "
48     )
49

```

3. METHODOLOGY

```
50 # -----
51 # Mesh 1
52 # -----
53 apply_thickness(
54     "mesh1",
55     known_pos=[
56         [point1x, point1y, point1z],
57         [point2x, point2y, point2z],
58         # ... additional sample coordinates
59         [pointN_x, pointN_y, pointN_z],
60     ],
61     known_val=[
62         thickness1,
63         thickness2,
64         # ... additional thickness values
65         thicknessN,
66     ]
67 )
68
69 # -----
70 # Mesh 2
71 # -----
72 apply_thickness(
73     "mesh2",
74     known_pos=[
75         [point1x, point1y, point1z],
76         [point2x, point2y, point2z],
77         # ... additional sample coordinates
78         [pointM_x, pointM_y, pointM_z],
79     ],
80     known_val=[
81         thickness1,
82         thickness2,
83         # ... additional thickness values
84         thicknessM,
85     ]
86 )
```

Listing 3.1: Interpolation of the thickness distribution in Blender using inverse distance weighting.

One limitation of this choice should be stated openly, because it also explains a later decision. IDW here uses straight-line (euclidean) distance through space, not distance measured along the curved surface. Across a fold, or across the gap between the two leaflets, straight-line distance could mix values that really should not be mixed, because two points can be close in space while being far apart along the tissue. This is exactly why the two leaflets are kept as two separate meshes and interpolated separately: a value measured on one leaflet cannot leak across the gap onto the other. Within a single,

fairly flat leaflet the difference between straight-line distance and along-surface distance is small enough to accept for the purpose of this work.

3.7 Building the solid model

The last step has this problem to solve: there is now a thickness value at every vertex, and a single surface that represents the underside of the leaflet, and this has to be turned into a closed solid whose wall has the correct local thickness.

For this step, Blender’s Solidify modifier was used [Ble26]. This modifier takes a surface and extrudes it by a chosen thickness to produce a solid, and, importantly, it can scale that thickness per vertex using a vertex group. This is what makes it fit the situation here: the interpolated thickness field from Section 3.6 is stored as exactly such a vertex group, so it maps directly onto how far each part of the surface is pushed out. In practice the modifier’s thickness is set to the largest thickness value, given in metres because Blender works in metres, and the zero-to-one weights then scale every other part of the surface below that maximum. The local thickness at a vertex is the weight at that vertex multiplied by the modifier thickness.

The extrusion is done in one direction only, which is where the check from Section 3.2 finally pays off. Because the mesh is the underside of the leaflet and not the midsurface, the whole thickness has to be added on one side, growing outward from the surface, rather than split half on each side. This is also the concrete reason why the “two times the distance” rule from the distance-field idea in Section 3.3 would have been wrong for this data: a factor of two is only correct when the surface is the centre, and here it is a boundary. In the modifier this is set by using a one-sided offset so that the material grows only outward. The direction of the extrusion follows the surface normals, so the normals were checked beforehand and flipped where needed, to make sure the wall grows away from the leaflet body and not into it.

Each leaflet is solidified on its own before the two are combined. The reason is that the two leaflets have different thickness ranges, and a single modifier can only hold one thickness value and one vertex group. So [Mesh1] and [Mesh2] are each given their own modifier with their own maximum thickness. Only after the thickness has been fixed into the geometry, by applying the modifier, are the two meshes joined into one object. Joining them earlier would force a single shared thickness on both, and would also let the interpolation bleed across the gap between them, for the reason given in Section 3.6.

The output of this step is a single solid mesh in which the leaflet walls have the measured, locally varying thickness.

3.8 Printing the complete model

Once a sufficiently accurate approximation, or model, of the mitral valve had been successfully developed, the actual objective and core of this work was addressed: the fabrication

of a realistic ultrasound phantom. As accuracy and realism were prioritized over ease of production and scalability, FDM 3D printing was selected as the manufacturing method [MCZS19]. Modern 3D printers provide sufficient detail resolution and, importantly for this project, the ability to work with flexible materials such as TPU. The fabrication of flexible structures using molds and a material such as silicone would also have been possible and could even have enabled scalable production. However, 3D printing was ultimately chosen because it allows for easy modifications to the model and eliminates the need to consider how the mold can be removed after casting.

Throughout the project, while the thickness measurements were being determined, test prints were conducted in parallel to identify the approach that would yield the best possible result. It was apparent that the heart valve would require support structures during printing, as the convex geometry would need to rest as flat as possible on the build plate, resulting in overhangs of well above 60° [MCZS19]. Printing the model vertically slightly reduced the cross-sectional area requiring support, but not to a significant extent. Furthermore, this orientation caused the structure to become unstable during printing, as the build plate moves back and forth along the Y-axis. The higher the print progressed, the more pronounced this movement became, causing the structure to wobble.

A major disadvantage of support material is the difficulty of completely removing all residues and achieving a smooth surface after printing. Test prints using PLA, for example, which is a relatively rigid plastic, demonstrated that removing the support structures alone was already a considerable challenge. Achieving a surface as smooth as the upper side of the print would then require substantial additional effort. Printing with flexible TPU did not significantly change this situation. The TPA95A used was specified with a Shore A hardness of 95; in general, lower Shore A values correspond to softer and more flexible TPU materials [reca]. It could be printed using settings that were almost identical to those used for PLA. For TPU82A [recc], the printing speed was reduced to avoid compromising print quality. However, at the desired thickness of the heart valve, this material remained too inflexible. TPU60A [recb], which provided an appropriate degree of flexibility and softness, introduced another problem: the amount of support material used previously was insufficient to adequately support the material between the individual support structures. The resulting print contained numerous holes and areas where the printed material had fused with the supports. The solution was to increase the density of the support structures by reducing the spacing between them. In this case, this was achieved by selecting smaller squares in the grid pattern.

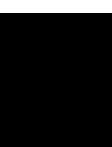
This resulted in a print using the desired material that appeared intact at first glance. However, the issue of the difficult-to-remove support structures remained. To address this problem, additional prototypes were produced using a multi-nozzle printer capable of printing with multiple materials simultaneously. The idea was to print the supports using a water-soluble filament, in this case polyvinyl alcohol (PVA) [MKM⁺23], while continuing to print the mitral valve using TPU60A. The finished model could then have been placed in water, allowing the supports to dissolve completely without leaving any residue.

However, there were concerns that the TPU might not adhere sufficiently to the PVA supports during printing, potentially resulting in print defects in the model. Therefore, PLA supports were initially used instead. Ultimately, this proved to be the right choice for this particular application, as the PLA and TPU adhered strongly enough to ensure a successful print while still allowing the valve to be easily peeled away from the support material after completion. As hoped, this resulted in a perfectly smooth underside. Furthermore, the gap within the valve opening was preserved.

The only remaining step was to manually remove any excess strands of filament and create a small incision in the opening using a scalpel. This was necessary because 3D printing small openings generally leaves behind thin strands of filament, a phenomenon known as *stringing*, which can cause the opening to become partially sealed. The completed prototype, including the support structures, was handed over to Siemens Healthineers, where the valve is currently undergoing further testing.

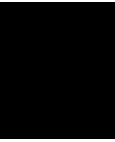
change if Siemens
gets back with
results

CHAPTER 4



Results

CHAPTER 5



Conclusion

Overview of Generative AI Tools Used

Ihr Text hier.

List of Figures

3.1	Planes added to the mesh in Blender to correspond to the bounding box of the volume.	8
3.2	Locations of the manual thickness measurements on the leaflets meshes. .	10
3.3	Top-down view of the leaflet mesh in Blender, showing the interpolated thickness values as a vertex group. The color gradient represents the thickness, with red indicating thicker regions and blue indicating thinner regions. . .	12

List of Tables

3.1	Applied leaflet thickness and 3D spatial coordinates from the Blender model at all sampled locations, outer rim points, and center points for Mesh1 and Mesh2.	11
-----	--	----

List of Algorithms

Bibliography

- [Ble26] Blender Foundation. Solidify modifier. Blender 5.2 LTS Manual, 2026. Accessed: 2026-08-27.
- [edM] Morphology - Distance Transform — [homepages.inf.ed.ac.uk. https://homepages.inf.ed.ac.uk/rbf/HIPR2/distance.htm](https://homepages.inf.ed.ac.uk/rbf/HIPR2/distance.htm). [Accessed 26-08-2026].
- [FBKC⁺12] Andriy Fedorov, Reinhard Beichel, Jayashree Kalpathy-Cramer, Julien Finet, Jean-Christophe Fillion-Robin, Sonia Pujol, Christian Bauer, Dominique Jennings, Fiona Fennessy, Milan Sonka, et al. 3d slicer as an image computing platform for the quantitative imaging network. *Magnetic resonance imaging*, 30(9):1323–1341, 2012.
- [HMVDW⁺20] Charles R Harris, K Jarrod Millman, Stéfan J Van Der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with numpy. *nature*, 585(7825):357–362, 2020.
- [MCZS19] Hugo I Medellin-Castillo and Jorge Zaragoza-Siqueiros. Design and manufacturing strategies for fused deposition modelling in additive manufacturing: a review. *Chinese Journal of Mechanical Engineering*, 32(1):1–16, 2019.
- [MKM⁺23] Mahmoud Moradi, Mojtaba Karamimoghadam, Saleh Meiabadi, Shafqat Rasool, Giuseppe Casalino, Mahmoud Shamsborhan, Pranav Kattungal Sebastian, Arun Poullose, Abijith Shaiju, and Mohammad Rezayat. Optimizing layer thickness and width for fused filament fabrication of polyvinyl alcohol in three-dimensional printing and support structures. *Machines*, 11(8):844, 2023.
- [reca] Filaflex 95A TPU Filament is compatible with all 3d printers — [recreus.com. https://recreus.com/en/products/filaflex-95a?variant=45921633829123](https://recreus.com/en/products/filaflex-95a?variant=45921633829123). [Accessed 27-08-2026].

- [recb] TPU Filament Filaflex 60A, flexible filament for 3d printing — recreus.com. <https://recreus.com/en/products/filaflex-60a?variant=45921624555779>. [Accessed 27-08-2026].
- [recc] TPU Filament Filaflex 82A, Flexible Filament for 3D Printing — recreus.com. <https://recreus.com/en/products/filaflex-82a?variant=45921644773635>. [Accessed 27-08-2026].
- [She68] Donald Shepard. A two-dimensional interpolation function for irregularly-spaced data. In *Proceedings of the 1968 23rd ACM National Conference*, pages 517–524. Association for Computing Machinery, 1968.